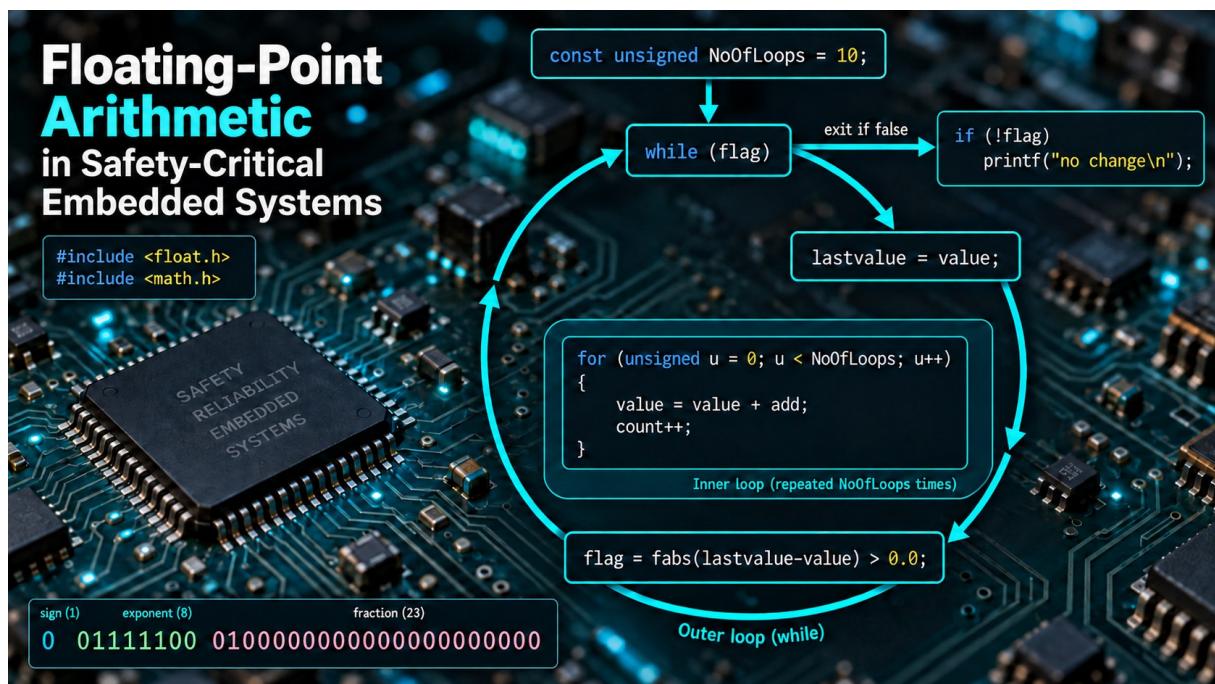


Floating-Point Arithmetic in Safety-Critical Embedded Systems

A practical assurance strategy for float32, model-based development, and ISO 26262 and IEC 61508



Bernhard Wagner | Computer Science Graduate, MBA

Automotive | Functional Safety | Model-Based Development | Quality Management

With round-to-nearest rounding, floating-point accumulation stagnates when a positive increment is smaller than half the distance to the next larger representable value. For float32 with an increment of `add=0.001f`, this occurs at `32768.0f`. Further additions leave the value unchanged regardless of the number of further executions. The comparison then sets `flag` to false and terminates the loop.

For example, a battery-management system may estimate accumulated charge by repeatedly adding small increments/decrements to a capacity counter. With float32 arithmetic, adding `0.001f Ah` stops changing the counter once it reaches `32768.0f Ah`. The accumulated rounding error can exceed the specified error budget long before stagnation occurs.

1. Abstract

- a) Floating-point arithmetic is not inherently unsafe. The safety issue is uncontrolled numerical behavior and insufficient evidence that the implemented calculation remains valid over its specified operating range.
- b) A float32 decision must be supported by explicit error budgets, range analysis, algorithmic stability analysis, and evidence from the production toolchain and target hardware.
- c) Model-in-the-loop results using double precision cannot by themselves justify code generated for a single-precision target. Software-in-the-loop and processor-in-the-loop comparisons are essential whenever precision, evaluation order, compiler optimization, or FPU behavior can change the result.
- d) IEEE 754 exception flags and processor traps are useful diagnostic mechanisms, but their behavior is target-specific. Inexact results are routine and must not be treated like invalid operations, divide-by-zero, overflow, or safety-relevant underflow without an architectural rationale.
- e) Simple universal limits - for example, a fixed maximum number of operations or a blanket ban on recursion - are not substitutes for a justified numerical analysis. Project-specific rules must be derived from the required accuracy, input ranges, dynamics, and fault response.

Core assurance claim: For every safety-relevant calculation, demonstrate that valid inputs produce bounded outputs with sufficient accuracy on the actual target implementation, and that detectable numerical failures lead to a defined reaction before the safety requirement can be violated.

2. Introduction

Safety-critical embedded software increasingly relies on model-based development, automatic code generation, and microcontrollers with hardware floating-point units. These technologies can improve development speed, clarity, and performance. They also create a recurring assurance question: how can an engineering team show that a calculation designed and simulated in one numerical environment remains sufficiently accurate, stable, and deterministic when deployed as float32 code on an embedded target?

The underlying challenge is not that floating-point arithmetic is categorically inferior to fixed-point arithmetic. Each representation has strengths and constraints. Fixed-point makes scaling, resolution, saturation, and overflow visible in the design. Floating-point provides a wide dynamic range and often maps naturally to control-engineering models, but its resolution changes with magnitude and its arithmetic does not preserve all real-number algebraic identities.

3. The engineering trade-off: fixed point or floating point

The choice of numeric representation should be treated as a design decision with explicit assumptions and acceptance criteria. A preference based only on execution speed, legacy practice, or model convenience is insufficient for a safety-related function.

Decision aspect	Floating-point arithmetic	Fixed-point arithmetic
Dynamic range	Very wide; resolution varies with magnitude.	Limited by word length and scale; resolution is explicit and uniform within the selected scaling.
Implementation	Often concise and natural for physical models; depends on FPU, compiler, and library behavior.	Requires scaling, saturation, and conversion logic; can be efficient and predictable.
Typical numerical risks	Rounding, cancellation, non-associativity, exceptional values, unsupported subnormals, and implementation differences.	Overflow, underflow, quantization, saturation, scaling errors, and intermediate-range growth.
Verification focus	Error propagation, stability, exceptional conditions, target equivalence, and FPU diagnostics.	Range, scaling, saturation, quantization, overflow paths, and conversion boundaries.

Decision aspect	Floating-point arithmetic	Fixed-point arithmetic
Safety argument	Valid when numerical behavior is bounded and demonstrated for the production implementation.	Valid when scaling and range assumptions are complete and overflow or saturation is safely handled.

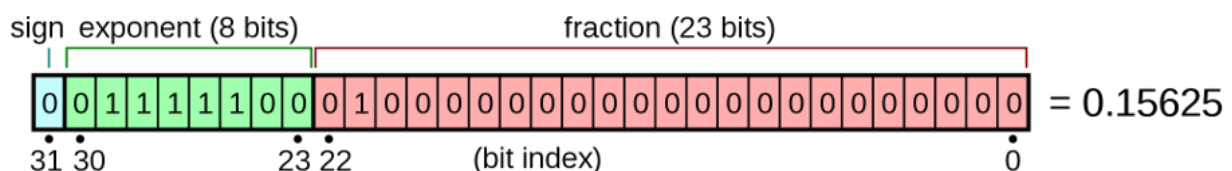
Decision rule: Select the representation that enables the clearest and most complete safety argument for the function. Mixed designs are legitimate: fixed-point may be preferable at interfaces or counters, while carefully controlled floating-point may be appropriate for filters, estimators, or physical models.

4. Why float32 calculations can surprise engineers

4.1. Finite representation and magnitude-dependent spacing

IEEE 754 binary32 uses one sign bit, an eight-bit exponent, and a 23-bit stored fraction. For normal numbers, the implicit leading bit gives 24 bits of significand precision. This corresponds to roughly six to nine significant decimal digits, depending on the conversion and value. The critical practical point is that adjacent representable numbers move farther apart as magnitude increases.

Example float 32 Number 0.15625



Binary32 field	Width	Engineering meaning
Sign	1 bit	Determines the sign of the represented value.

Binary32 field	Width	Engineering meaning
Exponent	8 bits	Provides the wide dynamic range through a biased exponent.
Fraction	23 stored bits	Together with the implicit leading bit, provides 24 bits of precision for normal values.
Special encodings	Exponent-dependent	Represent zero, infinities, NaNs, and subnormal values where supported.

At around 16,384, binary32 spacing is 0.001953125. Consequently, adding `float32(1.00097656)` to `float32(16383.0)` produces `float32(16384.0)`, while the exact real-number sum is approximately 16384.00097656. The overall result looks plausible, but the fractional part of the smaller correction has disappeared. Whether this matters depends on the function's error budget and on what the correction represents.

4.2. Cancellation

Cancellation occurs when nearly equal quantities are subtracted. The leading significant digits cancel, so the remaining result may contain far fewer trustworthy digits than either operand. The phenomenon is especially dangerous when the small difference is the information the algorithm actually needs.

Example 1: How the addition(1.00097656f+16383.000f≈16384.000f) works and why it fails

Let

S = SIGN

E = EXPONENT

[0..23] = Position in the fraction, the MSB is always '1' for normal numbers and not stored

Then we have the binary representation of both numbers as follows

	S	EEEEEEEE	1234567890	1234567890	123
1.00097656f =	(0	01111111	(1)	0000000001	0000000000 000 ₂)
16383.00000000f =	(0	10001100	(1)	1111111111	1110000000 000 ₂)

If we add two values in the form $a \cdot 2^n$ and $b \cdot 2^m$ the exponent n and m need to be the same. This is done by dividing the smaller value by 2 and increasing the exponent by one until $n=m$. Please note $b \cdot 2^m = (b/2) \cdot 2^{(m+1)}$

1) (Now adjust the exponent from 01111111₂ to 10001100₂)

	S	EEEEEEEE	1234567890	1234567890	123	
1.00097656f =	(0	01111111	(1)	0000000001	0000000000 000 ₂)	->0 (initial)
1.00097656f =	(0	10000000	(0)	1000000000	1000000000 000 ₂)	->1
1.00097656f =	(0	10000001	(0)	0100000000	0100000000 000 ₂)	->2
1.00097656f =	(0	10000010	(0)	0010000000	0010000000 000 ₂)	->3
1.00097656f =	(0	10000011	(0)	0001000000	0001000000 000 ₂)	->4
1.00097656f =	(0	10000100	(0)	0000100000	0000100000 000 ₂)	->5
1.00097656f =	(0	10000101	(0)	0000010000	0000010000 000 ₂)	->6
1.00097656f =	(0	10000110	(0)	0000001000	0000001000 000 ₂)	->7
1.00097656f =	(0	10000111	(0)	0000000100	0000000100 000 ₂)	->8
1.00097656f =	(0	10001000	(0)	0000000010	0000000010 000 ₂)	->9
1.00097656f =	(0	10001001	(0)	0000000001	0000000001 000 ₂)	->10
1.00097656f =	(0	10001010	(0)	0000000000	1000000000 100 ₂)	->11
1.00097656f =	(0	10001011	(0)	0000000000	0100000000 010 ₂)	->12
1.00097656f =	(0	10001100	(0)	0000000000	0010000000 001 ₂)	->13

2) With the aligned significands we can now add the fraction

1.00097656f =	(0	10001100	(0)	0000000000	0010000000 001 ₂)
16383.0000f =	(0	10001100	(1)	1111111111	1110000000 000 ₂)
				(+)	1 ₂ +1 ₂ = 10 ₂ we have a carry)
	(0	10001100	(10)	0000000000	0000000000 001 ₂)

-> shift right once more and increase the exponent by one

	(0	10001101	(1)	0000000000	0000000000 000 1 ₂)
--	----	----------	-----	------------	---------------------------------

-> (the 1 is no longer in the mantissa what is the issue here)

16384.0000f = (0 10001101 (1) 0000000000 0000000000 000₂) -> (We have lost 1/1024 accuracy of the addend)

Using float32 arithmetic with round-to-nearest, ties-to-even rounding, adding 1.00097656f to 16383.000f produces 16384.000f. The rounding error is approximately $1/1024 \sim 0.098\%$ of the addend. This may be unacceptable depending on the use case.

4.3. Feedback, recursion, and accumulated error

When an updated value is fed back into the next execution cycle, small rounding errors can accumulate, decay, or be amplified according to the algorithm's dynamics. This is common in integrators, filters, observers, counters, state estimators, and adaptive functions. A loop that adds a small delta to an increasingly large state can eventually reach a point at which the delta is below the local spacing and the state no longer changes.

The correct engineering response is not a universal prohibition on feedback or recursion. It is a stability argument: define the operating horizon, model the error propagation, establish bounds, use a numerically suitable formulation, and verify the behavior at critical ranges and durations.

Example 2: Calculation of the Variance

The variance of a set of n values $x_1, x_2, \dots, x_n$ is defined as

$$s_n^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (1)$$

with

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (2)$$

An alternative and theoretically equivalent formula to compute the variance is given by

$$S_n^2 = \frac{1}{n} \left(\sum_{i=1}^n x_i^2 - \frac{1}{n} (\sum_{i=1}^n x_i)^2 \right) \quad (3)$$

$$S_{n+1}^2 = \frac{1}{n+1} \left(\sum_{i=1}^n x_i^2 + x_{n+1}^2 - \frac{1}{n+1} (\sum_{i=1}^n x_i + x_{n+1})^2 \right) \quad (4)$$

The advantage for computing is that this formula (3) needs only one pass over the data and can be easily extended with a new element x_{n+1} , if you have stored $\sum_{i=1}^n x_i^2$ and $\sum_{i=1}^n x_i$ in variables.

For the test data $x_n = (10000.0f, 10001.0f, 10002.0f)$, the Formula (1) gives the correct population variance **$s^2 = 2/3 \approx 0.6666667f$** ,

$$s^2 = \frac{(10000 - 10001)^2 + (10001 - 10001)^2 + (10002 - 10001)^2}{3} = \frac{2}{3} \approx \mathbf{0.6666667}$$

whereas Formula (3) gives (**$s^2 = 0.0f$**) when evaluated using float32 arithmetic.

$$\sum x_i = 10000 + 10001 + 10002 = 30003 \Rightarrow \frac{1}{3} \left(\sum x_i \right)^2 = \frac{30003^2}{3} = 300060003$$

$$\sum x_i^2 = 100000000 + 100020001 + 100040004 = 300060005$$

$$s^2 = \frac{300060005 - 300060003}{3} = \frac{2}{3} \approx \mathbf{0.6666667}$$

using float32 this becomes

$$s^2 = \frac{300060000.0f - 300060000.0f}{3.0f} = \frac{0.0f}{3.0f} \approx \mathbf{0.0f}$$

In the illustrated float32 evaluation, both corresponding computed terms round to 300060000.0f, so their subtraction produces zero.

The computed result may also depend on the evaluation order, compiler optimizations, the precision of intermediate results, and the use of fused operations.

4.4. Rounding and invalid algebraic transformations

Finite-precision arithmetic is generally not associative or distributive. Reordering operations, replacing a division with multiplication by a reciprocal, enabling fused multiply-add, or changing compiler optimization can alter the result. Zero multiplication also requires care because IEEE 754 includes infinities and NaNs: an expression that is harmless over finite values may behave differently after an exceptional value has entered the data path.

1. Do not assume $(a + b) + c$ equals $a + (b + c)$ in float32.
2. Do not assume $a * (b + c)$ exactly equals $a * b + a * c$.
3. Do not treat compiler reassociation, fast-math options, or fused operations as semantics-preserving without analysis.
4. Define the permitted handling of NaN, infinity, subnormal values, signed zero, and rounding modes.

5. Functional-safety perspective

5.1. ISO 26262 Safety of electrical and electronic (E/E) systems in road vehicles

ISO 26262 does not impose a blanket ban on floating-point arithmetic. Its system- and software-level framework requires safety requirements, architectural measures, implementation constraints, verification, and evidence appropriate to the assigned ASIL and the identified failure modes. For model-based development, numerical accuracy, data types, discretization, target behavior, and toolchain assumptions therefore belong in the safety argument whenever they can affect a safety requirement.



The relevant question is not whether the source model ran successfully on an engineering workstation. The relevant question is whether the production software, compiled with the released settings and executed on the selected microcontroller, satisfies its accuracy and timing requirements over all safety-relevant operating conditions.

Standards status: The clause references in the 2024 source analysis relate to ISO 26262:2018. As of September 7, 2026, ISO/DIS 26262-6, Edition 3, is under development and is intended to replace the 2018 edition. Projects must use the edition required by contract and monitor the revision for future process impacts.

5.2. IEC 61508: Safety of electrical, electronic, and programmable electronic safety systems

IEC 61508-7 describes techniques and measures supporting the IEC 61508 framework, including offline numerical analysis. The engineering principle is straightforward: numerical inaccuracies can arise from finite representations, rounding, and approximation, and calculations must be checked to confirm that the accuracy required by the application is actually achieved. This reinforces the need for explicit numerical analysis rather than reliance on test results alone.



5.3. A practical assurance strategy

A credible strategy combines prevention, analysis, implementation controls, independent verification, and runtime detection. The following sequence can be integrated into Automotive SPICE and ISO 26262 development activities without creating a separate parallel process.

Step	Required engineering activity	Principal evidence
1. Define	Specify ranges, units, resolution, accuracy, timing, operating horizon, and safe reaction.	Quantitative software and technical safety requirements.
2. Analyze	Identify cancellation, feedback, singularities, overflow/underflow, exceptional values, and sensitive transformations.	Numerical risk analysis linked to model elements and requirements.
3. Design	Select stable algorithms, data types, scaling, saturation, guard conditions, and diagnostic mechanisms.	Architecture and model/coding guidelines with rationale.
4. Implement	Control code generation, libraries, compiler options, FPU configuration, and initialization.	Baselined toolchain and reproducible build configuration.

Step	Required engineering activity	Principal evidence
5. Verify	Perform boundary tests, equivalence-class tests, long-duration tests, robustness tests, and MIL/SIL/PIL comparisons.	Traceable reviews, analysis results, and test reports.
6. Monitor	Detect relevant exceptions or plausibility violations and transition to a defined degraded or safe state.	Diagnostic concept, fault-injection results, and safety mechanism verification.

5.4. Requirements and error budgets

A statement such as 'the calculation shall be accurate' is not verifiable. Define the allowed absolute and relative error, the input domain, expected transients, update rate, maximum execution duration, and the conditions under which the function must enter a degraded or safe state. Allocate the error budget across sensor uncertainty, conversion, algorithmic approximation, discretization, numerical representation, and implementation.

- Specify units and valid ranges for every input, state, calibration, and output.
- Define behavior outside the valid range, including clamping, rejection, substitution, degradation, or shutdown.
- State whether the accuracy criterion is absolute, relative, or both, especially near zero.
- Define the time horizon over which accumulated error must remain bounded.

5.5. Static and model-level analysis

Analyze equations and model paths before relying on simulation. Group expressions into equivalence classes with comparable numerical behavior but ensure that each class is justified by its operator sequence, ranges, feedback structure, and target data types. Review tools should flag risky constructs and retain traceability to the model and requirement.

Numerical hazard	Analysis question	Typical control
Large magnitude ratio	Can a small correction disappear when added to a large state?	Rescale, center, accumulate in higher precision, or restructure the state.
Cancellation	Are nearby large values subtracted to obtain a small result?	Use a stable formulation, compensated method, or centered variables.
Feedback	Can roundoff accumulate or be amplified over time?	Bound error dynamics; test long durations and worst-case sequences.
Singularity	Can a denominator approach zero or a function leave its mathematical domain?	Range guards, plausibility checks, fallback behavior, and diagnostics.
Exceptional values	Can NaN, infinity, subnormal, overflow, or underflow enter or propagate?	Define target behavior, configure detection, and verify the safety reaction.
Transformation	Can code generation or optimization reorder or fuse operations?	Control options; compare generated code and target results against an oracle.

5.6. MIL, SIL, and PIL verification

Model-in-the-loop, software-in-the-loop, and processor-in-the-loop results answer different questions. A high-precision model can serve as a reference oracle, but it does not reproduce all target effects. The comparison should use quantitative tolerances derived from requirements, not an arbitrary global epsilon.

1. MIL: validate the intended algorithm and identify sensitivity using high precision and float32 configurations where possible.
2. SIL: verify generated or production-equivalent code, data types, libraries, evaluation order, and compiler behavior.

3. PIL/target tests: confirm FPU configuration, exceptions, timing, processor-specific limitations, and binary reproducibility.
4. Long-duration and sequence testing: exercise accumulated error, drift, repeated saturation, and rare transitions.

5.7. Target FPU behaviour and exceptions

The FPU is part of the safety-relevant implementation and must be analyzed like any other hardware/software execution resource. Processor families differ in supported formats, handling of subnormal numbers, fused operations, exception flags, trap timing, library functions, and errata. The released microcontroller variant and toolchain documentation are therefore configuration items for the safety case.

IEEE 754 condition	Typical meaning	Safety-engineering treatment
Invalid operation	Undefined operation or invalid operand combination; often produces NaN.	Detect where safety-relevant; prevent propagation; apply defined fault response.
Divide by zero	Finite nonzero value divided by zero; often produces infinity.	Use input guards and, where justified, exception detection or trapping.
Overflow	Rounded result exceeds the finite representable range.	Treat as range violation; prevent unsafe output and transition as specified.
Underflow	Tiny result loses precision or becomes subnormal/zero depending on implementation.	Assess whether loss of a small value can affect the safety function; verify target behavior.
Inexact	Rounded results differ from the infinitely precise result.	Expected in normal floating-point operation; monitor only with a specific diagnostic rationale.

For the Infineon AURIX TC3xx example referenced by the source analysis, official documentation describes processor-specific FPU behavior, exception flags, and optional trap generation. Infineon's guidance states that trap

generation is disabled by default and controlled through FPU_TRAP_CON. This is a configuration fact, not a complete safety concept: the project must decide which conditions are relevant, how asynchronous traps interact with the FTTI, and whether a trap, status flag, plausibility check, or diversified computation provides the required coverage.

5.8. Self-test and diagnostic coverage

A power-on or periodic FPU self-test can be a useful safety mechanism when it addresses an identified fault model and provides sufficient diagnostic coverage within the fault-tolerant time interval. However, executing one example of each mathematical operation does not automatically demonstrate complete FPU integrity, and ISO 26262 does not turn that example into a universal requirement. The test concept should identify the faults it detects, its independence, its coverage, its execution frequency, and the reaction to a failed test.

Assessment point: Do not claim 'state of the art' merely because another project uses a core self-test. Demonstrate why the selected mechanism is suitable for this item: this ASIL, this processor, this fault model, and this diagnostic interval.

5.9. Verification criteria and work products

The assurance process should produce reviewable evidence rather than a collection of informal calculations. Evidence must be traceable to safety requirements, model elements, generated code, target configuration, tests, findings, and accepted residual limitations.

Work product	Minimum content
Numeric representation decision	Alternatives, performance constraints, safety impact, assumptions, and approval.
Numerical requirements	Ranges, units, accuracy, timing, duration, invalid-input handling, and safe reaction.
Numerical risk analysis	Hazards, affected expressions, operating conditions, severity, controls, and traceability.

Work product	Minimum content
Model and coding rules	Permitted data types, casts, libraries, optimization settings, guards, and exceptional-value policy.
Toolchain baseline	Model tool, code generator, compiler, libraries, flags, processor variant, FPU mode, and relevant errata.
Verification report	MIL/SIL/PIL comparison, boundary and robustness tests, long-duration tests, tolerance rationale, and coverage.
Diagnostic verification	Injected condition, expected detection, timing, reaction, coverage argument, and pass/fail evidence.
Open issues and deviations	Finding, safety impact, owner, due date, rationale, approval, and closure evidence.

5.10. Acceptance checklist

- ☐ Every safety-relevant numeric path is linked to requirements and valid ranges.
- ☐ Accuracy and stability criteria are quantitative and include the operating duration.
- ☐ Sensitive formulas, cancellation, feedback, and exceptional values have been analyzed.
- ☐ Project heuristics are identified as project-specific and supported by calculation or test evidence.
- ☐ Generated code, compiler settings, libraries, FPU configuration, and processor errata are baselined.
- ☐ MIL/SIL/PIL deviations are explained and remain within requirement-derived tolerances.
- ☐ Exception handling and diagnostics meet the specified reaction time and are verified by fault injection where appropriate.
- ☐ Residual numerical limitations are documented, reviewed, and reflected in the safety case.

6. Conclusion

Floating-point arithmetic can be used responsibly in safety-critical embedded systems, but convenience is not evidence. The engineering team must understand how finite precision, algorithm structure, code generation,

compiler optimization, processor behavior, and runtime diagnostics interact with the safety requirements.

The strongest argument is layered:

- quantitative requirements define what must be preserved;
- numerical analysis identifies weak points;
- stable design reduces sensitivity;
- the production toolchain and target configuration are controlled;
- MIL, SIL, and PIL verification demonstrate equivalence within justified tolerances; and
- runtime mechanisms detect failure modes that cannot be prevented.

This turns float32 from an implicit implementation choice into an explicit and auditable safety decision.

7. Sources, Liability and publication status

This article is based on the author's technical analysis and is provided for general informational purposes. Its conclusions must not be applied directly to safety-related systems without an application-specific assessment, verification, and validation. Although the content has been prepared with due care, the author assumes no liability for its accuracy, completeness, or use.

Standards references and processor guidance were checked against the following official sources as of September 7, 2026. The full standards are available from their publishers.

- [1] [IEEE 754-2019, Standard for Floating-Point Arithmetic](#)
- [2] [ISO 26262-4:2018, Product development at the system level](#)
- [3] [ISO 26262-6:2018, Product development at the software level](#)
- [4] [ISO/DIS 26262-6, Edition 3, under development](#)
- [5] [IEC 61508-7:2010, Overview of techniques and measures](#)
- [6] [MathWorks, Floating-Point Numbers](#)
- [7] [MathWorks, Single-Precision Format](#)
- [8] [Infineon, AURIX TC3xx Family User's Manual Part 1](#)
- [9] [Infineon, Generating traps on divide-by-zero exception in the FPU](#)

8. List of abbreviations

Abbreviation	Meaning
ASIL	Automotive Safety Integrity Level
ECU	Electronic Control Unit
FPU	Floating-Point Unit
FTTI	Fault-Tolerant Time Interval
MBD	Model-Based Development
MIL	Model-in-the-Loop
NaN	Not a Number
PIL	Processor-in-the-Loop
SIL	Software-in-the-Loop; in IEC 61508 context also Safety Integrity Level

9. About the Author

Bernhard Wagner, Diplom-Informatiker and MBA, is a functional safety and systems engineering specialist with more than 25 years of experience in software, electronics, systems development, and quality management. His work focuses on applying ISO 26262 and IEC 61508 to safety-critical embedded systems, including hazard and risk analysis, safety concepts, safety requirements, system and software architecture, traceability, and verification and validation. He also has experience with EN ISO 13849-1 and IEC 62061 in industrial environments.



Throughout his career, Bernhard has worked as a software developer, system architect, project manager, functional safety specialist, and technical director. He combines engineering expertise with extensive knowledge of Automotive SPICE, helping organizations integrate functional safety into standards-compliant development and quality-assurance processes. Bernhard has participated in ASAM and ISO working groups and is a co-author of ISO 22900-2 and ISO 22900-3.